

Offline-first, part 2: conflicts and CRDTs

Ordering events without a clock, resolving edits, and data that cannot conflict

Dinu-Ștefan Rusu

FILS, Universitatea Politehnica București · Spring 2027

What you should leave with



Order events honestly

Why a wall clock cannot order two edits, and what a Lamport or vector clock gives you instead.



Pick a strategy

Last writer wins, first writer wins, merge by field, ask the user, and what each one costs.



Ship a resolver

One Dart function that both devices run and that both devices agree on.



Read a CRDT

Counters, registers and sets whose merge is commutative, associative and idempotent.

PART 1 · CONFLICTS

Two devices editing the same row

Where a conflict comes from, and why your clock cannot settle it

Where Lecture 8 left off



The local database is the truth

The UI reads a Drift query. Every synced row carries id, updatedAt, deletedAt, pendingSync and serverVersion.



Every write goes to an outbox

The row and the intent to sync it commit in one transaction, so the queue never drifts out of step with the data.



Delta sync on a checkpoint

Push the outbox oldest first, pull the rows changed since the cursor the server issued.

One line was left as a placeholder. In the pull loop, when an incoming row touches a row this device also edited, we called `resolve(local, remote)`. This lecture writes it.

Where conflicts actually come from

A conflict is not a race in your code. It is two edits to the same row, made while neither side could see the other.

Offline-first guarantees they happen: accept the write now, reconcile later.

One user with a phone and a laptop is enough.

Update and update

Both devices set the title of the same note. The case everybody thinks of.

Update and delete

One deletes the row, the other edits it. No value satisfies both users.

Create and create

Two devices create the same record. Only a natural key catches this.

Device clocks cannot order events

Every device believes its own clock. Drift between time syncs, a user setting the time by hand, a cheap real-time clock: seconds of disagreement is normal, minutes is not rare.

REAL TIME	DEVICE	ITS CLOCK READS	THE EDIT
t = 0	Phone, 12 s fast	09:14:07	title becomes “Milk”
t = +40 s	Laptop, 40 s slow	09:13:55	title becomes “Oat milk”

The later write carries the earlier timestamp. Last writer wins keeps “Milk”. The newer edit goes with no error, no log and no undo, whenever two clocks disagree by more than the gap between two edits.

What ordering we actually need



Happens-before

Event a happens-before b if a is earlier on the same device, or a is a message send and b its receipt. Then take the transitive closure.



Concurrent

If neither a happens-before b nor b happens-before a, the two events are concurrent. That is the formal name for a conflict.



Not real time

Happens-before says nothing about which edit came first by the wall clock. It only says which edit could have seen the other.

If one edit could have seen the other, the newer one is an informed correction and should win. If neither could, no rule is right for both users, and you have to choose a policy in advance.

PART 2 · ORDERING

Order without a clock

Lamport counters, vector clocks, and the practical compromise

Lamport clocks: a counter, not a time

```
class Lamport {  
    int l = 0;  
    int local() => ++l;    // any local  
        event  
    int send() => ++l;    // stamp the  
        message  
    void receive(int theirs) {  
        l = (theirs > l ? theirs : l) + 1;  
    }  
}
```

Three rules, one integer.

Increment on a local event. Attach the counter to a message. On receipt, take the maximum and add one.

One way only: if a happens-before b, then $L(a)$ is smaller.

The converse is false, and that is the problem. It does not prove causality: the two events may simply have been concurrent.

Vector clocks: one counter per replica

```
typedef Vc = Map<String, int>;      // replica id -> its own counter
Vc tick(Vc v, String me) => {...v, me: (v[me] ?? 0) + 1};
Vc join(Vc a, Vc b) =>
    {for (final k in {...a.keys, ...b.keys}) k: max(a[k] ?? 0, b[k] ?? 0)};
/// -1: a before b. 1: b before a. 0: equal. null: concurrent.
int? compare(Vc a, Vc b) {
    final ks = {...a.keys, ...b.keys};
    final aLe = ks.every((k) => (a[k] ?? 0) <= (b[k] ?? 0));
    final bLe = ks.every((k) => (b[k] ?? 0) <= (a[k] ?? 0));
    return aLe && bLe ? 0 : aLe ? -1 : bLe ? 1 : null;
}
```

Ordering is component-wise, so null comes back exactly when the two edits are concurrent.

A worked example: two devices, one note

STEP	WHERE	P CLOCK	L CLOCK	WHAT HAPPENS
0	both	{P:0, L:0}	{P:0, L:0}	last common state, in sync
1	phone	{P:1, L:0}	{P:0, L:0}	offline, edits the title
2	laptop	{P:1, L:0}	{P:0, L:1}	offline, edits the body
3	sync	compare returns null		neither vector dominates
4	resolve	{P:2, L:1}	{P:0, L:1}	merge by field, P bumps its slot
5	laptop	{P:2, L:1}	{P:2, L:1}	L is dominated: take it

P is the phone, L the laptop. Step 5 is the payoff: L does not guess, its vector is smaller in every slot.

Hybrid logical clocks, the compromise

```
class Hlc implements Comparable<Hlc> {  
    final int wallMs;    // ms since the epoch  
    final int counter;   // ties inside one ms  
    final String node;   // replica id, tiebreak  
  
    Hlc tick(int nowMs) => nowMs > wallMs  
        ? Hlc(nowMs, 0, node)  
        : Hlc(wallMs, counter + 1, node);  
  
    Hlc receive(Hlc r, int nowMs) =>  
        Hlc(max(nowMs, max(wallMs, r.wallMs)),  
            _tieBreak(r, nowMs), node);  
}
```

Vector clocks cost state per device.

The vector grows with every replica that ever existed: on a consumer app, every phone the user ever owned.

An HLC is 64 bits on the wire.

The wall part keeps the value readable. The counter part never allows it to go backwards, so the order never contradicts causality.

Use it as your updatedAt.

PART 3 · STRATEGIES

Four ways to resolve, honestly ranked

Each one loses something. Decide what, in advance and in writing

Four strategies, honestly ranked

STRATEGY**HOW IT DECIDES****WHAT IT COSTS****REACH FOR IT WHEN**

Last writer wins	highest stamp takes the whole row	silent loss: nobody is told	one device at a time, low value fields
First writer wins	the server keeps the row it has	a rejection you must show	claims, invites, created-once rows
Merge by field	column by column, changed fields only	one resolver per entity	documents with independent fields
Ask the user	keep both, show a chooser	UI work and an interruption	anything expensive to lose

Most apps need two of these: merge by field for documents, first writer wins for money.

Last writer wins

```
Note resolve(Note l, ServerNote r) =>
    l.updatedAt.isAfter(r.updatedAt)
    ? l
    : fromServer(r);
```

One comparison, no per-entity code. That is the entire appeal.

The example.

A note reads “Buy milk”. On the phone the user fixes the title to “Buy oat milk”. On the laptop, a minute later, the same user adds three lines to the body.

The laptop row is newer, so the whole row wins. The title correction is gone.

The unit of loss is the row, not the field. Use it for one low value column, such as a read flag or a sort position. Never for text the user typed.

First writer wins

The example.

A shared shopping list. Two housemates, both offline in the same supermarket, both create “bread” under the same natural key: list id plus lowercased name.

The first push to reach the server creates the row. The second gets a 409 back, drops its copy and adopts the server’s id. The list has one bread.

The part people forget

The losing device already showed the write to the user. When the server rejects it, the app has to fix the screen quietly and keep anything that was typed.

First writer wins needs a natural key. Without one the two creates are simply two different rows, and the rule never fires. A unique constraint on the server enforces it, not a timestamp.

Merge by field

The same note and the same two offline edits. The row now stores a stamp per field, not one stamp for the whole row.

FIELD	PHONE VALUE	STAMP	LAPTOP VALUE	RESULT
title	“Buy oat milk”	09:14	“Buy milk”	phone wins the field
body	unchanged	09:02	three new lines	laptop wins the field
tags	{shopping}	08:40	{shopping}	unchanged on both sides

The unit of loss shrinks from the row to the column. The cost is one resolver per entity plus a stamp per field, roughly eight bytes with an HLC. Test it with local and remote swapped: the same output, or the devices never converge.

Ask the user

The example.

Two people edit the description of the same product while offline. Both texts are long and deliberate, and no field-level rule can pick between them.

Keep both. Store the remote version as a conflict row, mark it in the list, and open a two-pane chooser when the user taps it.

Never block the app

The marker sits on one row. Everything else stays usable while it waits.

Resolve, then push

The choice is a new write with a new stamp, so it goes through the outbox like any edit.

Ask rarely, and only where losing the text would hurt. A dialog on every sync trains the user to tap the first button.

A resolver you can ship

```
Note resolve(Note local, ServerNote remote) {  
  if (!local.pendingSync) return fromServer(remote); // they are newer  
  if (remote.version <= local.serverVersion) return local; // we are ahead  
  return local.copyWith( // both sides moved past the same version  
    title: _pick(local.title, local.titleAt, remote.title, remote.titleAt),  
    body: _pick(local.body, local.bodyAt, remote.body, remote.bodyAt),  
    deletedAt: local.deletedAt ?? remote.deletedAt, // delete wins  
    serverVersion: remote.version, pendingSync: true, // push it back  
  );  
}
```

`_pick` keeps the larger HLC. Push the merged row back, or the two sides never converge.

PART 4 · CRDTS

Data types that cannot conflict

Move the decision out of the resolver and into the type

Convergence without coordination

A conflict-free replicated data type, or CRDT, is a type whose merge is defined so that replicas holding the same updates hold the same state, with no leader and no round trip.



Commutative

$\text{merge}(a, b)$ equals $\text{merge}(b, a)$.
Updates may arrive in any order, over any path.



Associative

Grouping does not matter, so replicas can gossip in whatever topology the network gives them.



Idempotent

$\text{merge}(a, a)$ equals a . The same update twice is a no-op, which makes retries free.

Convergent is not the same as correct. A CRDT guarantees every replica agrees. It cannot enforce “the balance never goes below zero”, because that needs coordination by definition.

One slide of the underlying math

The state space is a join-semilattice.

Order states by information: $x \leq y$ means y contains every update x contains, and possibly more.

Merge is the least upper bound.

$x \sqcup y$ is the smallest state containing both: nothing invented, nothing dropped.
Concurrent states are simply incomparable.

Why this converges

Every update moves the state upwards: $s \leq u(s)$. The join is commutative, associative and idempotent, so two replicas holding the same updates compute the same bound.

Pick the order first, and the merge follows. Sets ordered by inclusion merge with union. Integers ordered by size merge with maximum. Maps merge slot by slot.

State-based versus operation-based

	STATE-BASED	OPERATION-BASED	DELTA-STATE
What travels	the whole state	one operation	the fragment that changed
Delivery	anything, even lossy	exactly once, causally ordered	anything, even lossy
A duplicate	harmless: merge is idempotent	corrupts the value	harmless

Prefer state-based or delta-state on a phone. A delta is a state in the same lattice, so a lost one still converges. Operation-based CRDTs push their hard problem into the transport.

A counter that survives being merged

A plain integer cannot merge. Phone says 4, laptop says 3: is the answer 4, or 7? Keep the history: one slot per replica, each incrementing only its own.

STATE	PHONE SLOT	LAPTOP SLOT	TABLET SLOT	VALUE = SUM
Phone holds	3	1	0	4
Laptop holds	1	2	0	3
Merge, max per slot	3	2	0	5
Tablet then adds 1	3	2	1	6

The merge is a maximum, not an addition. That is why receiving the same state twice changes nothing. The value is the sum of the slots.

The grow-only counter in Dart

```
class GCounter {  
  final Map<String, int> slots;      // replica id -> that replica's count  
  const GCounter(this.slots);  
  int get value => slots.values.fold(0, (a, b) => a + b);  
  GCounter inc(String me) => GCounter({...slots, me: (slots[me] ?? 0) + 1});  
  GCounter merge(GCounter o) => GCounter({  
    for (final k in {...slots.keys, ...o.slots.keys})  
      k: max(slots[k] ?? 0, o.slots[k] ?? 0)});  
}
```

The map is the whole trick, and the value is derived. Generate the replica id once per install and never reuse it: two devices sharing an id lose increments.

Decrements, and a register

PN-Counter

Two grow-only counters, subtracted.

```
class PnCounter {  
    final GCounter p, n; // plus, minus  
    int get value => p.value - n.value;  
    PnCounter inc(String me) =>  
        PnCounter(p.inc(me), n);  
    PnCounter dec(String me) =>  
        PnCounter(p, n.inc(me));  
    PnCounter merge(PnCounter o) =>  
        PnCounter(  
            p.merge(o.p), n.merge(o.n));  
}
```

LWW-Register

One value, one stamp. Merge keeps the larger stamp.

```
class LwwReg<T> {  
    final T value;  
    final Hlc at;  
    LwwReg<T> merge(LwwReg<T> o) =>  
        at.compareTo(o.at) >= 0  
            ? this : o;  
}
```

Merge by field is a map of these, one per column.

Sets: add and remove do not commute



G-Set, grow-only

State is a set, merge is union.
Converges perfectly and can
never remove anything.



2P-Set, two phases

One set of adds, one of
removes. Present when in adds
and not in removes. Union both
on merge.



The re-add problem

Once an element reaches the
removed set it is dead forever.
Adding “milk” again does
nothing.

Apply add(x) then remove(x), then the reverse order. The two answers differ. A set that supports both has to give every add its own identity.

The observed-remove set

Every add gets a unique tag. An element is present when at least one of its tags is still live.

A remove takes away only the tags this replica observed. It cannot touch a tag it never saw.

Merge is the union of the adds and the union of the removed tags.

💡 One worked case

A adds “milk” as tag a1. B sees a1 and removes “milk”, so removed is {a1}. A, concurrently, adds “milk” again as a2. Merge: adds are {a1, a2}, removed is {a1}, so a2 survives.

Add wins, because the remover could not have observed it. A defensible default for a shopping list and the wrong one for a bank ledger. It is a policy chosen by the type, not a law.

The observed-remove set in Dart

```
class OrSet<E> {  
  final Map<E, Set<String>> adds = {};    // element -> its unique tags  
  final Set<String> removed = {};         // tags seen as removed  
  Set<E> get value => {for (final e in adds.entries)  
    if (e.value.difference(removed).isEmpty) e.key};  
  void add(E e) => adds.putIfAbsent(e, () => {}).add(uuid.v4());  
  void remove(E e) => removed.addAll(adds[e] ?? const {}); // observed  
  void merge(OrSet<E> o) {  
    o.adds.forEach((e, t) => adds.putIfAbsent(e, () => {}).addAll(t));  
    removed.addAll(o.removed);}  
}
```

Both fields only grow.

Merge is union twice.

remove reads the local adds.

Only tags seen here go.

PART 5 · COSTS

Tombstones, growth, and limits

What convergence costs in bytes, and when to use something else

Deleted is a row, not a missing row

Delete locally and the next pull cannot tell “deleted here” from “not reached me yet”. The server sends it back and the item reappears.

So a delete is a write: set `deletedAt`, keep the row, filter it out of every UI query. The tombstone is what propagates the deletion.

Collecting the garbage

A tombstone is safe to drop only once every replica has seen the deletion. Proving that needs every device to report what it saw, and one device offline for months holds up the whole set.

Pick the retention window on day one. Purge tombstones older than, say, 90 days, and make a device offline longer than that do a full resync instead of a delta.

What a CRDT costs in space

1 slot

per replica in a counter, forever,
even for a device sold years ago

1 tag

per add in an OR-Set, plus one
entry per observed removal

2x

roughly, a merge-by-field row
against a plain row

In a long-lived collaborative document the tombstones can outnumber the live elements by a large factor. Text CRDTs keep a node for every character ever typed, deleted ones included.

The metadata is the product. A CRDT buys convergence by storing enough history to make every merge deterministic. If you cannot afford the history, you cannot have the guarantee.

When a CRDT is the wrong tool



A global invariant

Stock must never go below zero. That needs coordination, which is what a CRDT gives up.



The user must be told

A refund or a booking. Converging silently is worse than a rejection.



The shape does not fit

An ordered list, or a tree with a move, is not a set.



One writer at a time

If one device edits it, merge by field is smaller and simpler.



The server can decide

A server-authoritative rule is one comparison and no metadata.



Nobody can maintain it

A merge bug converges confidently on the wrong value, unseen by single-device tests.

What to use in your project

YOUR DATA	USE	WHY
A sensor reading	append only	nothing ever updates the row
A note or a form	merge by field on an HLC	the columns are independent
A view count	PN-Counter	retries are free
A shared tag list	OR-Set	add and remove both have an answer
A booking	first writer wins, on the server	one machine holds the invariant

The project asks for a stated rule, not a clever one. Lab 5 in week 10 builds the outbox, the drain loop and delta sync. Name your rule per entity in the README, and let the demo show it firing.

Three things to remember

1. Wall clocks cannot order events. Lamport counters respect causality, vector clocks also detect concurrency, and an HLC is the compromise that fits in 64 bits.
2. Every strategy loses something. Last writer wins loses a row, merge by field loses a column, first writer wins loses a rejection you must show, asking costs an interruption.
3. A CRDT converges, and that is all it promises. The merge must be commutative, associative and idempotent, the metadata never shrinks on its own, and no CRDT holds a global invariant.

FURTHER READING

[crdt.tech](#), [the catalog and the papers](#) · [inkandswitch.com/local-first](#) · [hybrid logical clocks, explained](#)

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel